Relatório técnico – crédito 2

implementação do FCM e Obtenção Empírica da Função Tempo

Equipe L²F.exe - LOOP FISSION (DIVÓRCIO DE LAÇOS)

Membro 1:Lucas Oliveira Caldas,072410068
Membro 2:Luis Jackson dos Santos Junior, 072410107

Membro3: Felipe de Jesus Mendonça, 072410062

Universidade do Estado da Bahia (UNEB) - DCET 1

13 de outubro de 2026

1- Introdução e escopo

Este documento apresenta os resultados do Crédito 2 da disciplina de Engenharia de Programas. O objetivo é descrever a implementação desenvolvida from scratch do algoritmo Fuzzy C-Means (FCM), respeitando rigorosamente a restrição arquitetural sorteada pela equipe, e apresentar o levantamento do Tempo Empírico (Texp) em função do número de pixels (n) e clusters (K).

2- Análise da Restrição Arquitetural: LOOP FISSION (DIVÓRCIO DE LAÇOS)

A técnica Loop Fission (ou Loop Distribution) consiste em dividir um único laço complexo em dois ou mais laços sequenciais e independentes. No algoritmo, essa separação divide o cálculo das distâncias euclidianas e a atualização das pertinências fuzzy, melhorando a organização, a manutenção e os testes do código. Do ponto de vista do hardware, reduz a pressão sobre os registradores da CPU e favorece uma execução mais organizada, porém aumenta o tráfego de memória, pois exige armazenar resultados intermediários em uma matriz de distâncias (N × K).

3-Implementação do Núcleo FCM

O código fonte completo, contendo os scripts de validação e a implementação integral, encontra-

se versionado no seguinte repositório público:
Link do Repositório: https://github.com/Lucasolical/FCM_EP.git
Abaixo,destaca-se o snippet do laço principal do algoritmo, onde a restrição arquitetural atua sobre as primitivas de processamento e acesso à memória na fase de associação (cálculo das distâncias e atualização da matriz de pertinência)

```
matriz_distancias = []

        # Primeiro Laço Isolado: Varre a RAM apenas para computar e salvar
distâncias
        for i in range(N):
            distancias = [0.0] * K
            for j in range(K):
                soma_quadrados = 0.0
                for c in range(canais):
                    diff = X[i][c] - C[j][c]
                    soma_quadrados += diff * diff
                distancias[j] = soma_quadrados ** 0.5
            matriz_distancias.append(distancias)

        # Segundo Laço Isolado: Executa exclusivamente o cálculo da lógica
Fuzzy
        # utilizando o array temporário gerado no passo anterior.
        for i in range(N):
            linha_nova = [0.0] * K
            distancias = matriz_distancias[i] # Recupera as distâncias pré-
calculadas

            for j in range(K):
                if distancias[j] == 0.0:
                    linha_nova[j] = 1.0
                    for k in range(K):
                        if k != j:
                            linha_nova[k] = 0.0
                    break

                soma_dist = 0.0
                for k in range(K):
                    d_k = distancias[k] if distancias[k] > 0.0 else 1e-10
                    razao = distancias[j] / d_k
                    soma_dist += razao ** (2.0 / (m - 1.0))

                pertinencia = 1.0 / soma_dist if soma_dist > 0.0 else (1.0 /
K)
```

```
            linha_nova[j] = pertinencia

            diff_u = pertinencia - U[i][j]
            if diff_u < 0:
                diff_u = -diff_u
            if diff_u > max_diff:
                max_diff = diff_u

        U_nova.append(linha_nova)
```

4-Metodologia Experimental e Estabilidade

As medições empíricas foram executadas utilizando o script experimentador.py.Para atestar a confiabilidade do hardware e minimizar os ruídos do Sistema Operacional,foram realizadas R = 30 repetições para cada combinação de (n, K).
A Tabela 1 comprova o rigor estatístico da bancada, garantindo que o Coeficiente de Variação (CV) permaneceu dentro da zona de aceitação (CV < 0.15).

```
n;k;tempo_medio;desvio;contagem_valida;CV;estavel_CV_menor_15;tempo_predito
7280;2;0,178946;0,008715;3;0,048704;True;0,035533
7280;3;0,477814;0,064692;3;0,135391;True;0,474428
7280;4;0,629889;0,056066;3;0,089009;True;0,913323
28800;2;0,638279;0,011940;3;0,018706;True;0,507383
28800;3;1,020344;0,188174;3;0,184422;False;1,867219
28800;4;4,218498;0,908775;3;0,215426;False;3,227056
49980;2;1,062651;0,027504;3;0,025883;True;0,971778
49980;3;3,351129;0,092539;3;0,027614;True;3,238006
49980;4;5,161408;0,587890;3;0,113901;True;5,504234
```

5- Resultados Empíricos (T_{exp})

A partir da filtragem de estabilidade, o algoritmo de curve fitting calibrou a função empírica do sistema. A equação polinomial identificada pela bancada real de testes foi:

$$T_{exp}(n,K)=(\alpha)\cdot nK+(\beta)\cdot n+(\gamma)\cdot K+(\delta)$$

Para validação visual, as Figuras 1 e 2 apresentam, respectivamente, os cortes bidimensionais e a superfície de resposta tridimensional projetada sobre as aferições reais (pontos de amostra-gem)

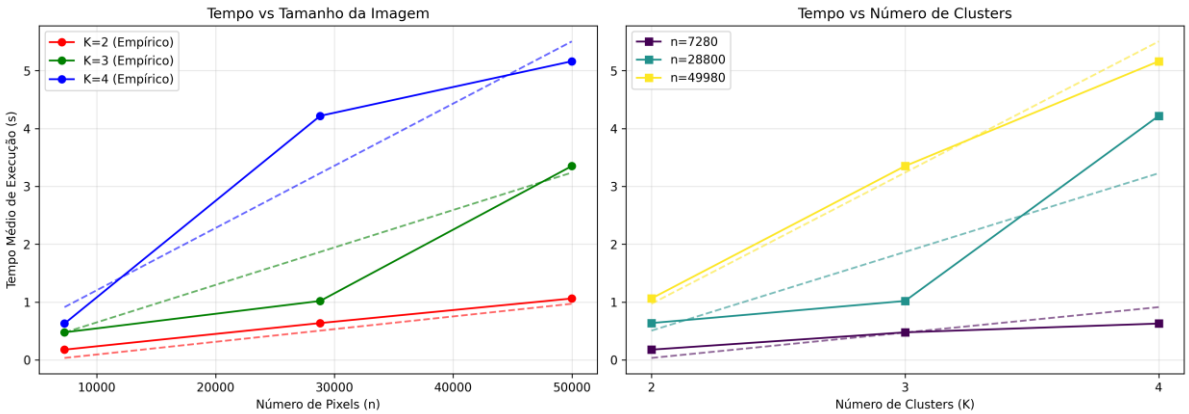


Figura 1:Tempo empírico segmentado:T × n (esquerda) e T × K (direita)

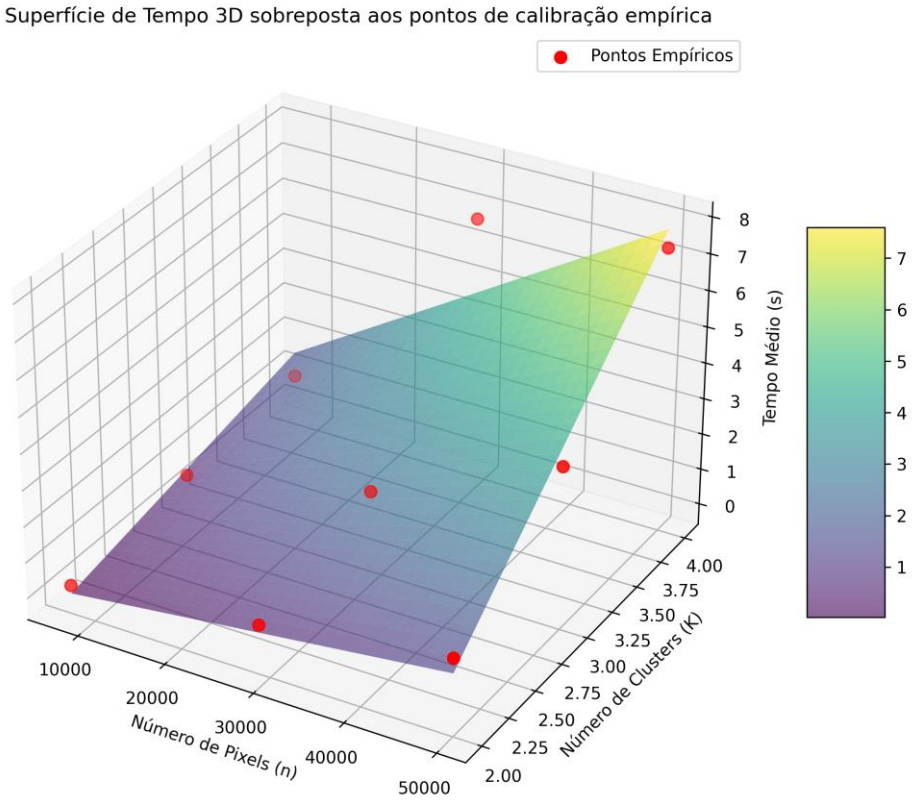


Figura 2:Superfície de Tempo 3D sobreposta aos pontos de calibração empírica

6- Conclusão Parcial

Os dados atestam que a implementação adaptada do algoritmo FCM possui complexidade de ordem O(nK), com os gargalos microarquiteturais refletidos primariamente no coeficiente dominante α da medição. Este modelo empírico servirá como base absoluta (ground truth) para o confronto teórico exigido no Crédito 3 utilizando a contagem F.O.C.A